

YourTodo 데이터 아키텍처 리포트

작성일: 2026-05-17 분석 기준: origin/main 최신 개선 버전 4683ff4 대상: YourTodo Android 앱 현재 코드베이스 범위: Android 앱 내부 데이터 모델, 저장소, 동기화, 네트워크, ViewModel 소비 흐름

1. 요약

현재 앱은 feature -> domain -> data -> storage/network 방향의 layered architecture를 따른다. 화면은 feature:* ViewModel이 core:domain use case와 repository contract만 바라보고, 실제 저장소와 네트워크 조합은 core:data repository 구현체가 맡는다. core:database, core:datastore, core:network는 각각 Room, Preferences DataStore, Retrofit 경계를 제공한다.

최신 main에는 이전 리포트의 남은 개선 후보가 다수 반영됐다. Todo sync는 categoryId, dueTimeMinutes까지 서버 왕복 대상에 포함하도록 확장됐고, TodoRepositoryImpl 내부 collaborator는 Hilt로 주입되는 @Singleton 구성으로 승격됐다. Assignment feed freshness는 Room row의 cacheUpdatedAt, in-memory tracker, DataStore refresh timestamp를 함께 합성한다. Workspace refresh는 single-flight에 더해 5분 stale threshold, forceRefresh, cached-result skip 정책을 갖췄다.

가장 큰 최신 업데이트는 PersonVisibility다. 사용자가 친구별 내 할일 보여주기 권한을 켜면 서버 grant와 Android visibility_grants cache가 동기화되고, 상대는 내 Todo 흐름을 ObservedTodo로 읽기 전용 확인한다. 이 데이터는 Shared Todo나 Direct Assignment와 섞이지 않고, Friends의 활성 친구 row와 Calendar의 조건부 친구 할일 섹션에서만 소비된다. Room은 version 13으로 올라가 visibility_grants, observed_todos, observed_sync_state를 갖게 됐고, app-level sync는 workspace refresh와 person visibility refresh를 병렬 실행한다.

보안과 실패 처리도 더 명확해졌다. Auth token cipher는 decrypt 결과와 failure type을 구조화했고, Keystore lookup 실패를 복구 가능한 failure로 분류한다. 새 auth session 저장에 실패하면 auth/session preference를 지우고, user-scoped local data는 보존해 재로그인을 유도한다. AppError 매핑은 stale local category/todo/reminder reference와 reminder repository 실패까지 더 넓게 적용됐다.

다만 새 개선으로 드러난 중요한 설계 과제도 있다. categoryId는 이제 Todo sync 필드지만 category 테이블 자체는 여전히 로컬 전용이고 todo.categoryId는 Room FK다. 즉 다른 기기 또는 서버에서 내려온 category id가 현재 기기의 category row와 일치하지 않을 수 있다. 새 PersonVisibility cache도 sign-out 시 user-scoped cleanup에 포함할지 명시해야 한다. 현재 UI는 auth session이 없으면 관찰 Flow를 비우지만, 로컬 DB row 자체는 currentUserId로 partition된 상태로 남을 수 있다.

2. 전체 계층 구조

```
app
- Application, MainActivity, Navigation host
- FCM, notification, WorkManager reminder, widget update wiring
- profile menu, sign-out orchestration, app-level sync entry
- feature*:entry + core:* 통합

feature*:impl / feature*:widget
- Auth / Todo / Calendar / Friends 화면과 Calendar widget 구현
- ViewModel이 UIState + Action + SideEffect 구조로 화면 상태 생성
- core:domain use case와 scheduler contract 호출

core:domain
- use case, repository contract, scheduler contract
- task surface, reminder recurrence, workspace refresh policy, error taxonomy
- Android storage/network 구현 없음

core:data
- repository implementation
- Room DAO, DataStore source, Retrofit data source 조합
- Entity/DTO/DataStore 값과 domain model 간 mapper
- Todo repository collaborator DI

core:database / core:datastore / core:network
- Room DB와 DAO
```

- Preferences DataStore, token encryption policy, assignment freshness keys
- Retrofit API, DTO, network data source

core:model

- Android 프레임워크 비의존 순수 모델

의존 방향은 안정적이다. core:*는 feature:*를 참조하지 않고, feature 구현은 data 구현체가 아니라 domain use case를 호출한다. 앱 셀은 런타임 통합을 위해 core:data, core:database, core:datastore에 접근하지만, feature 구현 세부는 feature:*:entry 경계를 통해 연결한다.

3. 데이터셋별 진실의 원천

데이터셋	Domain model	현재 주 저장소	서버 동기화	주요 소비자	최신 상태
개인 Todo	TodoItem	Room todo	있음, outbox + cursor	Todo, Calendar, Widget, reminder worker	title/date/status/priority/categoryId/dueTime sync
Todo mutation	TodoSyncPayload	Room todo_outbox	push API	Todo sync coordinator	Json DI, fallback fields, transaction runner 사용
Category	Category	Room category	없음	Todo editor/list	로컬 전용이지만 Todo sync가 categoryId를 왕복
범용 Reminder	Reminder	Room reminder	없음	Reminder use case	domain recurrence + AppError mapping
Todo reminder	TodoItem reminder 필드	Room todo + WorkManager	reminder 세부 필드는 미포함	Todo editor, worker	same recurrence calculator 사용
Auth session	AuthSession	DataStore encrypted token + profile keys	Auth API	Auth gate, authenticated repos	structured cipher/read failure
Todo filter/sort	TodoFilter, TodoPriorityFilter, TodoSortOption	DataStore	없음	Todo list	TodoFilterPreferences로 분리
Todo sync cursor	string cursor/halt reason	DataStore	sync API	Todo sync coordinator	auth-required halt 상태 유지
Push token	current/registered token	DataStore + Push API	있음	FCM service, registration VM	sign out 전에 server delete 시도
Friend	Friend, FriendRequest	서버 응답 + 화면 in-memory snapshot	REST API	Friends screen, assignment editor	initial unavailable / stale snapshot UX
Assigned todo	AssignedTodo, AssignmentBundle	서버 + Room cache + DataStore freshness	REST API	Todo, Calendar, Widget, Friends	feed refresh time 영속화
Person visibility grant	PersonVisibilityGrant	서버 + Room visibility_grants	REST API	Friends visibility switch	owner/viewer directional grant
Observed todo	ObservedTodo	서버 projection + Room observed_todos	observed sync API	Friends, Calendar	read-only friend todo, Todo/Widget 제외
AI Todo draft	AITodoDraftResult	저장 없음	AI proxy API	Todo AI draft sheet	stateless parse result

핵심은 데이터별 일관성 모델이 다르다는 점이다. 개인 Todo는 local-first와 서버 수렴을 동시에 추구한다. Friends는 persistent cache 없이 온라인 전용이지만, 이미 로드한 화면 snapshot은 stale banner와 함께 유지한다. Assigned todo는 서버가 진실의 원천이고 Room cache와 DataStore freshness timestamp를 함께 사용한다. Observed todo는 서버 grant가 접근권한을 결정하고 Android는 cursor 기반 read-only projection cache를 유지한다. Category와 reminder 세부값은 아직 Android 로컬 정책에 더 가깝다.

4. Room 데이터베이스

현재 AppDatabase는 version 13이며 다음 entity를 가진다.

Table	Entity	책임	주요 키/인덱스
todo	TodoEntity	개인 Todo, todo 내장 reminder, sync 상태	categoryId FK, isReminderEnabled/reminderAtEpochMillis, (ownerUserId, serverId), (ownerUserId, clientId), syncStatus
todo_outbox	TodoOutboxEntity	서버에 아직 전송하지 않은 Todo mutation queue	clientMutationId unique, todoLocalId, (ownerUserId, createdAt)
category	CategoryEntity	로컬 카테고리	name unique
reminder	ReminderEntity	범용 reminder	(isEnabled, triggerAtEpochMillis), updatedAt
assigned_todo	AssignedTodoEntity	받은/보낸 할당 Todo cache	(ownerUserId, id) primary key, feed/status/friend별 인덱스, cacheKey, cacheUpdatedAt
assigned_todo_checklist_item	AssignedTodoChecklistItemEntity	assigned todo checklist cache	(ownerUserId, assignedTodoId, id) primary key, cascade delete
visibility_grants	VisibilityGrantEntity	친구별 내 할일 보기 권한 cache	(currentUserId, grantId) primary key, (currentUserId, ownerUserId, viewerUserId) unique
observed_todos	ObservedTodoEntity	친구가 허용한 read-only Todo projection cache	(currentUserId, observedTodoId) primary key, owner/grant/sourceTodo 인덱스
observed_sync_state	ObservedSyncStateEntity	observed todo cursor와 sync 시각	currentUserId primary key

마이그레이션 흐름은 기능 확장 순서가 잘 드러난다. v1 Todo에서 시작해 Category, Reminder, Todo time/reminder/priority, Todo sync/outbox, assigned todo cache, direct assignment mode가 추가됐다. v11->v12 migration은 SQLite DDL이 없는 no-op이지만, 코드 주석으로 "direct-assignment task-surface cache policy 이후 exported schema 기록용 bump"라고 사유가 명시되어 있다. v12->v13 migration은 person visibility용 세 테이블과 인덱스를 추가하고, AppDatabaseMigrationTest가 컬럼/인덱스 생성을 검증한다.

DAO는 Flow 관찰을 기본으로 한다. Todo 목록과 월간 Todo는 deletedAt IS NULL 및 syncStatus != PENDING_DELETE 조건으로 soft-deleted/pending-delete 항목을 숨긴다. Assigned todo DAO는 received/sent, friend, status 조합의 feed cache를 관찰하며, freshness 판단을 위해 cacheUpdatedAt 기반 query도 제공한다. PersonVisibilityDao는 current user 기준 grant와 observed todo를 관찰하고, observed sync upsert/delete/purge/cursor 저장을 transaction으로 묶는다. Todo DAO에는 push fallback field 조회를 위한 getTodosByIds()와 sign-out owner cleanup용 deleteByOwner()가 있다.

5. Preferences DataStore와 token/freshness 저장

UserPreferencesDataSourceImpl은 user_preferences.pb에 앱 설정, 인증 메타데이터, 동기화 메타데이터, assignment freshness timestamp를 저장한다.

Key group	Keys	책임
Schema	user_preferences_schema_version	preference migration 버전
Auth encrypted	auth_access_token_encrypted, auth_refresh_token_encrypted	access/refresh token 암호화 저장
Auth legacy	auth_access_token, auth_refresh_token	migration 호환용 plaintext key
Auth profile	user id, nickname, email, onboarding flag	로그인 세션 복원
Todo UI	selected filter, category filter, priority filter, sort option	Todo list 선호 상태
Todo sync	sync cursor, halt reason	pull cursor와 auth halt 상태
Push	current token, registered token	FCM token 등록 상태
Assignment freshness	assignment_feed_refresh_time_*	feed별 마지막 refresh 시각

Token은 AuthTokenStoragePolicy만 통해 읽고 쓴다. 새 세션 저장은 AndroidKeyStoreAuthTokenCipher가 AES-GCM으로 암호화한 값을 DataStore encrypted key에 기록하고, legacy plaintext key를 삭제한다. 읽기는 encrypted token을 우선하며, encrypted decrypt가 실패해도 legacy plaintext token이 있으면 LegacyFallback으로 세션을 복원한다.

최신 구조는 실패를 더 세밀하게 분류한다. AuthTokenCipherFailure는 ENCRYPT, DECRYPT, KEY_LOOKUP, KEY_GENERATION operation과 INVALID_FORMAT, KEY_LOOKUP, KEY_GENERATION, ENCRYPTION, DECRYPTION, ENCODING failure type을 갖는다. Keystore lookup 실패는 key를 삭제하고 재생성할 수 있지만, 기존 encrypted token은 복호화 실패로 보고 legacy fallback 또는 재로그인이 필요해질 수 있다.

Auth session 저장 실패 정책도 명확해졌다. AuthRepositoryImpl은 sign-in/onboarding 후 secure save가 실패하면 auth/session preference를 지우고 예외를 다시 던진다. 이때 Todo/assigned cache 같은 user-scoped local data는 삭제하지 않는다. 반대로 명시적 sign out은 user-scoped local data, sync cursor/halt reason, assignment freshness timestamp까지 정리한 뒤 auth session을 지운다.

6. 네트워크 계층

core:network는 Retrofit + kotlinox serialization + OkHttp로 구성된다. 일반 서버 Retrofit은 BuildConfig.YOURTODO_SERVER_BASE_URL, AI 전용 Retrofit은 BuildConfig.YOURTODO_AI_SERVER_BASE_URL을 사용한다.

API 영역	Retrofit API	Endpoint 예시	인증	DataSource
Auth	AuthApi	api/auth/google, api/auth/refresh, api/users/me/onboarding	일부 Bearer	AuthNetworkDataSource
Todo sync	TodoSyncApi	GET/POST api/sync/todos	Bearer	TodoSyncNetworkDataSource
Friends	FriendApi	api/friends, api/friend-requests/*	Bearer	FriendNetworkDataSource
Assignments	AssignmentApi	api/assignment-bundles, api/assigned-todos/*	Bearer + 일부 Idempotency-Key	AssignmentNetworkDataSource
Person visibility	PersonVisibilityApi	api/person-visibility/grants, api/observed-todos/sync	Bearer + mutation Idempotency-Key	PersonVisibilityNetworkDataSource
Push	PushApi	api/push-token	Bearer	PushNetworkDataSource
AI draft	AiTodoDraftApi	ai/todo-drafts	없음	AiTodoDraftNetworkDataSource

Todo sync DTO는 이제 NetworkTodo와 NetworkTodoMutationPayload 모두 categoryId, dueTimeMinutes를 포함한다. 주석도 "reminder trigger/repeat/lead settings만 Android-local"이라고 좁혀졌다. 각 Retrofit data source는 기능별 auth-required exception을 발생시키고, core:data repository는 AuthSessionRefresher로 refresh token 재발급을 한번 시도한다.

Person visibility API는 grant metadata와 observed todo projection을 분리한다. GET api/person-visibility/grants는 given/received grant를 반환하고, create/revoke는 idempotency key를 사용한다. GET api/observed-todos/sync는 cursor와 optional date window를 받아 upsert items, deleted ids, purged grant ids, next cursor를 반환한다. Android는 이 응답을 Room transaction으로 적용해 revoke purge와 cursor 저장을 같은 경계에서 처리한다.

7. Todo 저장소와 동기화

7.1 구현 책임 분리와 DI

TodoRepositoryImpl은 네 개 domain contract를 구현하는 facade다.

Contract	제공 기능
TodoItemRepository	Todo observe/get/add/update/delete/toggle/sync

Contract	제공 기능
TodoCategoryRepository	Category observe/add/update/delete
TodoFilterRepository	Todo filter/category/priority/sort preference observe/update
TodoReminderRepository	active todo reminder 조회

내부 collaborator는 이제 대부분 @Inject/@Singleton으로 주입된다.

내부 collaborator	책임
TodoLocalTodoStore	Todo CRUD, toggle, local sync status 전이
TodoOutboxStore	CREATE/UPDATE/DELETE mutation 저장과 삭제
TodoSyncCoordinator	pull/push/pull sync orchestration, cursor 갱신, auth refresh retry
TodoSyncSessionProvider	현재 auth session과 onboarding 상태 확인
TodoCategoryStore	Category observe/add/update/delete, selected category 정리
TodoFilterPreferences	filter/category/priority/sort preference observe/update
TodoReminderReader	활성화된 Todo reminder 조회
TodoTransactionRunner	Room transaction 경계 추상화
TodoTimeProvider	테스트 가능한 현재 시각 provider
TodoSyncPayloadJson	outbox payload encode/decode용 Json

이전의 "내부 분리는 됐지만 repository 안에서 조립된다"는 한계는 해소됐다. 이제 collaborator 단위 테스트와 교체가 쉬워졌고, repository facade는 delegation과 logging에 집중한다.

7.2 개인 Todo sync 흐름

로그인하지 않았거나 온보딩이 필요한 사용자의 Todo는 LOCAL_ONLY로 저장된다. 로그인 사용자의 신규 Todo는 PENDING_CREATE와 clientId를 받고, outbox에 CREATE mutation이 생성된다. update/delete는 기존 sync 상태에 따라 outbox를 갱신하거나 local-only 삭제로 처리한다.

```

사용자 변경
-> Add/Update/Delete/Toggle use case
-> TodoLocalTodoStore가 Room transaction 안에서 todo 즉시 반영
-> 로그인 상태면 TodoOutboxStore가 mutation 저장
-> UI는 Room Flow로 즉시 갱신

syncTodos()
-> TodoSyncCoordinator.pull(cursor)
-> remote todo를 Room에 적용
-> TodoOutboxStore.pendingMutations push
-> mutation 결과 적용 및 outbox 제거/FAILED 처리
-> pull(latest cursor)
-> DataStore todoSyncCursor 업데이트

```

충돌 정책은 단순하고 예측 가능하다. local PENDING_DELETE는 remote 변경보다 우선 보존되고, local PENDING_UPDATE도 remote pull로 덮지 않는다. 서버에서 삭제된 remote todo가 오면 outbox를 지우고 Room row를 deleted 상태로 반영한다. mutation reject는 local todo를 FAILED로 표시하고 lastSyncError를 남긴 뒤 outbox row를 제거한다.

7.3 Sync field contract

Todo sync 계약은 코드와 docs/TRD-Todo-Sync-MVP.md 양쪽에 명시됐다.

TodoSyncFieldPolicy.syncedPayloadFields는 서버 왕복 대상 필드를 다음 일곱 개로 고정한다.

서버 sync 필드	Android source	정책
title	TodoEntity.title	push/pull 대상
description	Android 개인 Todo 모델에는 아직 없음	서버 계약에는 존재하지만 Android push에서는 미사용

서버 sync 필드	Android source	정책
dueDate	TodoEntity.dueDateEpochDay	YYYY-MM-DD date-only
status	TodoEntity.isDone	ACTIVE/COMPLETED, tombstone은 server DELETED
priority	TodoEntity.priority	LOW/MEDIUM/HIGH
categoryId	TodoEntity.categoryId	개인 Todo 카테고리 ID
dueTimeMinutes	TodoEntity.dueTimeMinutes	0..1439 minute-of-day

아래 reminder 세부 필드는 현재 로컬 전용이다.

로컬 전용 필드	이유
reminderAtEpochMillis	Todo 내장 reminder sync 계약 없음
isReminderEnabled	Todo 내장 reminder sync 계약 없음
reminderRepeatType	반복 reminder sync 계약 없음
reminderRepeatDaysMask	반복 reminder sync 계약 없음
reminderLeadMinutes	reminder lead sync 계약 없음

TodoSyncMappers는 remote todo에 categoryId 또는 dueTimeMinutes가 없으면 기존 local field를 보존한다. outbox push도 payload에 누락된 extended field가 있으면 현재 Room row에서 fallback fields를 읽어 채운다.

7.4 Category sync 리스크

가장 중요한 남은 설계 리스크는 category다. TodoEntity.categoryId는 Room FK이고 CategoryEntity는 여전히 로컬 전용이다. 그런데 Todo sync는 categoryId를 서버와 왕복한다. 따라서 서버에서 내려온 categoryId가 현재 기기의 category table에 없거나, 같은 숫자가 다른 카테고리를 가리킬 수 있다.

이 문제는 Todo sync 확장 자체의 가치를 낮추는 것은 아니지만, 여러 기기 sync를 목표로 하면 반드시 정책이 필요하다.

- Category도 서버 sync 대상으로 승격한다.
- 서버 category id와 local Room category id를 분리한다.
- remote categoryId가 local에 없으면 null 처리하거나 "미분류"로 fallback한다.
- remote category id 적용 전에 local category 존재 여부를 검증하고 실패를 AppError.LocalDataMissing 또는 sync reject로 표준화한다.

현재 문서 기준으로 이 항목은 P1 개선 후보로 남긴다.

8. Assignment, Friends, Auth, Push repository

8.1 AssignmentRepositoryImpl

Assignment는 서버가 진실의 원천이고 Android는 받은/보낸 assigned todo feed를 Room에 캐시한다.

observeReceivedAssignedTodos, observeSentAssignedTodos, friend별 observe API는 DataStore auth session의 user id를 기준으로 Room Flow를 구독한다. 네트워크 get 호출은 응답 items를 domain으로 매핑한 뒤 feed별 cache를 교체한다.

Freshness는 2차 개선까지 반영됐다. AssignmentRepository.observeFeedCacheFreshness(feed)는 다음 세 소스를 합성한다.

- persisted row freshness: retained row들의 cacheUpdatedAt
- DataStore feed refresh time: assignment_feed_refresh_time_<storageKey>
- in-memory tracker: 현재 프로세스에서 방금 refresh된 feed time

AssignmentFeedCacheFreshness는 기본 5분 stale threshold를 제공하고, isStale(now)와 shouldForceRefresh(now)로 호출자가 refresh 필요 여부를 판단할 수 있다. 빈 feed도 DataStore refresh time 덕분에 앱 재시작 후 freshness를 유지할 수 있다. Sign out은 AssignmentFeedFreshnessTracker.clear()와 clearAssignmentFeedRefreshTimes()를 함

깨 호출한다.

남은 작은 운영 리스크는 feed refresh key가 DataStore에 누적될 수 있다는 점이다. 현재 sign out에서는 정리되지만, 장기간 같은 계정에서 많은 friend feed 조합이 생길 경우 TTL/prune 정책을 고려할 수 있다.

8.2 PersonVisibilityRepositoryImpl

PersonVisibilityRepositoryImpl은 친구별 읽기 권한과 observed todo projection을 담당한다. 이 repository는 UserPreferencesDataSource.authSession을 기준으로 current user를 정하고, 로그인/온보딩 완료 session이 없으면 관찰 Flow를 빈 리스트로 돌린다.

기능	데이터 흐름
권한 관찰	Room visibility_grants -> PersonVisibilityGrant
권한 켜기	POST api/person-visibility/grants -> grant cache upsert
권한 끄기	active given grant 조회 -> DELETE api/person-visibility/grants/{grantId} -> grant revoked cache + observed rows purge
observed todo 관찰	Room observed_todos group by owner -> ObservedPersonTodos
grants refresh	network given/received grants -> Room replace
observed sync	Room cursor -> api/observed-todos/sync -> upsert/delete/purge/cursor transaction

Auth-required 응답은 다른 repository와 같은 패턴으로 AuthSessionRefresher를 한 번 사용하고, retry도 실패하면 auth session을 clear한 뒤 AuthRequiredException을 던진다. Revoke 성공 시 해당 grant의 observed rows는 즉시 purge한다.

이 구조의 의미는 ObservedTodo가 AssignedTodo나 내 Todo가 아니라는 점을 data layer에서부터 보존한다는 것이다. 별도 Room table, 별도 repository contract, 별도 network API를 사용하기 때문에 Todo/Assignment mutation 경로로 들어갈 수 없다. Calendar와 Friends는 use case를 통해 read-only projection만 소비한다.

남은 운영 리스크는 sign-out cleanup이다. AuthRepositoryImpl.deleteUserScopedLocalData()는 Todo, outbox, assigned todo cache와 sync/freshness state를 지우지만, 최신 코드 기준 person visibility rows와 observed_sync_state는 직접 삭제하지 않는다. auth session이 사라지면 Flow가 빈 리스트를 내보내 화면 노출은 막히지만, 기기 로컬 DB의 user-scoped row 보존 정책은 명시적으로 결정하는 것이 좋다.

8.3 FriendRepositoryImpl과 Friends stale UX

Friends와 friend requests는 여전히 온라인 전용이다. repository contract와 구현체 주석, docs/TRD-Friends-MVP.md가 "Room/DataStore 친구 캐시 없음, 실패를 빈 리스트로 취급하지 않음"을 명시한다.

화면 UX는 개선됐다. 최초 로드 실패 시 FriendsUnavailableBlock이 retry를 제공한다. 이미 한 번 로드한 friends snapshot이 있는 상태에서 refresh가 실패하면 기존 in-memory snapshot을 유지하되 FriendsStaleSnapshotBanner로 최신 상태가 아님을 표시한다. 즉 persistent offline cache는 없지만, "마지막으로 보던 화면을 갑자기 빈 목록으로 바꾸지 않는" UX는 들어갔다.

제품적으로 완전한 오프라인 friends 화면이 필요해지면 그때 Room cache와 stale timestamp가 필요하다. 현재는 온라인 전용 + in-memory stale UX로 의도가 분명하다.

8.4 AuthRepositoryImpl

Google id token으로 서버 로그인 후 AuthSession을 DataStore에 저장한다. 닉네임 온보딩 완료도 현재 access token으로 서버를 호출하고, 응답 user 정보를 DataStore에 다시 저장한다. Secure session 저장 실패 시 auth preference를 clear하고 실패를 반환한다.

명시적 sign out은 SignOutUseCase가 push token 삭제를 먼저 시도한 뒤 AuthRepository.signOut()을 호출한다. repository는 다음 정리를 수행한다.

- 현재 사용자 todo_outbox 삭제
- 현재 사용자의 모든 owner-scoped Todo 삭제
- 현재 사용자의 assigned todo cache 삭제

- Todo sync cursor/halt reason 삭제
- assignment feed freshness in-memory/DataStore 값 삭제
- Auth session 삭제

이전의 "synced Todo만 지워 local-only owner Todo가 남을 수 있음" 리스크는 `todoDao.deleteByOwner()`와 회귀 테스트로 정리됐다. 단, owner가 없는 비로그인 로컬 Todo는 별도 사용자 데이터가 아니므로 남을 수 있다.

8.5 PushTokenRepositoryImpl과 FCM

FCM token은 DataStore에 current token과 registered token으로 저장된다. 앱 시작 후 `PushTokenRegistrationViewModel`이 Firebase token을 받아 `RegisterPushTokenUseCase`를 호출하고, 로그인 user id 변화가 감지되면 기존 current token을 서버에 다시 등록한다.

`YourTodoFirebaseMessagingService`는 새 token 등록과 push 수신 처리를 담당한다. 메시지를 받으면 `RefreshWorkspaceUseCase`를 호출해 Todo/Friends/Assignment cache를 갱신하고, 로컬 메시지 포맷이 가능한 type은 앱 리소스 기반으로 알림 문구를 만든다. assigned todo title이 payload에 없으면 Room cache에서 title을 보강한다.

9. Domain 정책 계층

9.1 TaskSurface

Todo, Calendar, Widget이 공유하는 "사용자에게 보이는 할 일 표면" 정책은 domain use case로 이동해 있다.

Use case/model	책임
<code>TaskSurfaceItem</code>	local Todo와 assigned Todo를 같은 surface row로 표현
<code>TaskSurfaceSource</code>	local/assigned source 구분
<code>TaskSurfaceList</code>	items, sections, completed id set 제공
<code>TaskSurfaceSectionKey</code>	open/completed/priority/dueDate/friend/self 섹션 key
<code>BuildTaskSurfaceListUseCase</code>	Todo list filter/sort/section 구성
<code>BuildTaskSurfaceDateTodosUseCase</code>	Calendar selected date agenda 구성
<code>ObserveTaskSurfaceSummariesUseCase</code>	월간 local summary와 assigned todo summary 병합

최신 main은 이 정책의 테스트 matrix도 확장했다. 오늘 필터와 priority/sort 조합, assigned override, friend/self section, date summary merge, reminder summary 등 policy drift가 생기기 쉬운 케이스가 domain test로 고정됐다.

9.2 Reminder recurrence

반복 reminder 계산은 `core:domain/reminder/ReminderRecurrenceCalculator`로 통합됐다. 범용 `ReminderRepositoryImpl.completeReminder`와 app 모듈의 `TodoReminderWorker`가 같은 calculator를 사용한다.

지원 정책은 다음과 같다.

- NONE: 다음 trigger 없음
- DAILY: 현재 trigger 시각 기준 매일 반복하되 now 이후로 이동
- WEEKLY: 현재 trigger 시각 기준 매주 반복하되 now 이후로 이동
- CUSTOM_DAYS: bit mask 요일 중 now 이후 첫 후보 선택

최신 main은 recurrence test를 보강했고, `ReminderRepositoryImpl`은 failure를 `mapFailureToAppError()`로 normalize 한다.

9.3 AppError taxonomy

`core:domain/error`에는 앱 공통 실패 모델이 있다. `Throwable.toAppError()`, `Result.mapFailureToAppError()`, `AppErrorException` 계열을 통해 network timeout/connectivity, auth required, server validation, conflict, local data missing을 구분할 수 있다.

최신 main에서는 stale local reference 매핑이 강화됐다. `IllegalStateException`뿐 아니라 `IllegalArgumentException`도 “stale todo/category/reminder id/reference” 문맥이면 `AppError.LocalDataMissing`으로 분류한다. `Reminder repository failure`도 이 표준 mapping을 사용한다.

남은 과제는 adoption 범위다. 모든 repository와 feature UI가 이미 동일한 `AppError`를 소비하는 단계는 아니므로, error UI와 retry policy는 계속 맞춰가야 한다.

9.4 Workspace refresh policy

`RefreshWorkspaceUseCase`는 `@Singleton`이며 `Mutex + CompletableDeferred`로 in-flight refresh를 공유한다. 최신 main은 여기에 refresh policy를 추가했다.

요소	정책
<code>forceRefresh</code>	fresh cache가 있어도 강제 refresh
<code>allowCachedResult</code>	fresh full snapshot이면 네트워크 refresh skip 가능
<code>WorkspaceRefreshPolicy.STALE_THRESHOLD_MILLIS</code>	5분
partial snapshot	<code>isFullySynced=false</code> 면 skip하지 않고 다음 호출에서 refresh
widget update	refresh 성공 자체를 깨지 않는 best-effort 처리

실제 refresh는 `Todo sync`, `friends/request 조회`, `received assigned todo feed 조회`를 병렬 수행하고 snapshot을 `WorkspaceSyncNotifier`에 publish한 뒤 `calendar widget update`를 호출한다. 남은 과제는 process restart 이후에도 유지되는 workspace freshness, `WorkManager` 기반 retry/backoff, 배터리/네트워크 제약 반영이다.

9.5 Person visibility policy

`PersonVisibility`는 assignment와 다른 도메인 정책을 가진다. `SetVisibilityGrantUseCase`와 `RevokeVisibilityGrantUseCase`는 친구별 내 할일 보여주기 권한을 만들고 끄며, `ObserveVisibilityGrantsUseCase`는 owner/viewer 방향성을 가진 grant 상태를 화면에 제공한다. `ObserveObservedTodosUseCase`는 친구가 허용한 read-only todo projection을 owner별로 묶어 제공한다.

`RefreshPersonVisibilityUseCase`는 grants refresh와 observed todo sync를 병렬 실행한다. `AppSyncViewModel`은 기존 `RefreshWorkspaceUseCase`와 이 use case를 함께 실행하고, 두 결과가 모두 성공해야 사용자에게 전체 sync 성공을 보여준다. 즉 person visibility는 workspace sync의 일부처럼 경험되지만, data contract는 `Todo/Assignment`와 분리되어 있다.

10. 화면과 데이터 소비 흐름

10.1 Todo 화면

`TodoListViewModel`은 개인 `Todo Flow`, visible received assigned todo Flow, auth session, priority filter, sort option, local UI state를 combine한다. 최종 list/section/completed id 계산은 `BuildTaskSurfaceListUseCase`에 맡기고, `TodoListUiMapper`는 `TaskSurfaceItem`을 `TodoItemUiModel`로 변환한다.

쓰기 작업은 use case를 통해 수행한다. 추가/수정/삭제/완료 토글 이후 `reminder scheduler`, `calendar widget updater`, `Todo sync`를 후속 실행한다. Assigned todo 완료/재오픈/삭제는 `ManageAssignedTodoUseCase`를 통해 서버 mutation과 optimistic cache update를 사용한다.

10.2 Calendar 화면

`CalendarViewModel`은 월간 개인 `Todo`, visible assigned todo, observed todo, workspace sync snapshot을 조합한다. 월간 indicator는 `ObserveTaskSurfaceSummariesUseCase` 결과에 `mergeObservedTodoSummaries()`를 더해 local/assigned/observed indicator를 같은 달력 cell에 반영한다. selected date agenda는 `BuildTaskSurfaceDateTodosUseCase`의 내 할일/받은 할일 결과와 `buildObservedSelectedDateTodos()`의 친구 할일 결과를 합친다.

Calendar mapper는 month cell 구성, 날짜 normalize, 시간 label, source label 같은 UI 표현에 집중한다. Observed todo는 `CalendarTodoSource.FRIEND`로 표시되고, stable negative id와 `observed_<id>` item key를 사용해

owned/assigned id 공간과 충돌하지 않게 한다.

10.3 Friends 화면

FriendsViewModel은 friends, incoming/outgoing requests를 서버에서 조회한다. 최초 로드가 실패하면 unavailable block을 보여주고, 이미 로드한 snapshot이 있는 refresh 실패는 stale banner로 표시한다. Workspace refresh snapshot을 구독해 Friends 목록도 갱신한다.

Friend detail을 열면 assigned todo cache Flow를 observe하면서 서버 refresh를 수행한다. 할당 생성은 CreateAssignmentBundleUseCase로 서버에 보내고, 친구가 direct assignment를 허용한 경우 AssignmentMode.DIRECT, 아니면 REQUEST로 전송한다. pending decision은 received pending assigned todo를 기준으로 선택/accept/reject한다.

최신 Friends 화면은 PersonVisibility도 함께 소비한다. 활성 친구 row는 자동수락과 내 할일 보여주기를 별도 compact switch로 보여준다. observed todo가 있는 친구 row에는 친구 할일 n개 affordance가 나타나고, 사용자가 펼치면 해당 친구의 observed rows를 inline으로 보여준다. pending friend request row에는 visibility control이 나타나지 않는다.

10.4 Calendar Widget

Widget은 CalendarMonthWidgetPresenter가 CalendarMonthSummarySource를 통해 월간 summary를 가져온 뒤 widget state를 만든다. 실제 summary source는 domain ObserveTaskSurfaceSummariesUseCase를 사용한다. 따라서 Calendar 화면과 Widget이 같은 local+assigned summary 정책을 공유한다.

PersonVisibility MVP에서는 Widget이 observed todo를 읽지 않는다. Calendar 화면은 친구 할일을 조건부 섹션으로 보여줄 수 있지만, 홈 화면 widget은 공간과 개인정보 민감도를 고려해 내 Todo와 received assigned task surface만 표시한다.

GlanceCalendarWidgetUpdater는 domain CalendarWidgetUpdater 계약으로 바인딩되어 Todo 변경, workspace refresh 성공, reminder 상태 변경 뒤 widget update를 호출할 수 있다. Workspace refresh에서는 widget update 실패가 refresh result를 실패로 바꾸지 않게 best-effort 처리한다.

11. Workspace refresh와 동기화 트리거

```
RefreshWorkspaceUseCase(forceRefresh, allowCachedResult)
-> single-flight / cache skip decision
-> todoRepository.syncTodos()
-> friends / incoming requests / outgoing requests 병렬 조회
-> received assigned todos pending / active / history 병렬 조회
-> visible received assigned todos 계산
-> WorkspaceSyncNotifier.publish(snapshot)
-> CalendarWidgetUpdater.updateCalendarWidgets() best effort

RefreshPersonVisibilityUseCase(windowStart, windowEnd)
-> visibility grants refresh
-> observed todos cursor sync
-> Room visibility/observed cache 갱신

AppSyncViewModel.syncWorkspace()
-> RefreshWorkspaceUseCase와 RefreshPersonVisibilityUseCase 병렬 실행
-> 두 결과가 모두 성공해야 사용자 sync 성공으로 표시
```

호출 지점은 여러 곳이다.

- app sync 버튼 또는 화면 시작 흐름
- FCM message 수신
- Friends 화면 수동 refresh
- assignment complete/reopen/delete/cancel 성공 후
- Todo 화면 초기 quiet sync와 foreground sync
- Calendar/Friends 화면의 assigned todo refresh

single-flight와 5분 cached-result policy 덕분에 중복 실행 비용은 줄었다. 그래도 process-level memory 정책이므로 앱 재시작 후 workspace freshness는 다시 계산해야 한다. 장기적으로는 WorkManager backoff와 network constraint를 붙일 여지가 있다.

12. 최신 main에서 개선 완료된 항목

기존 개선 후보	현재 상태	근거
Token 저장 보안 강화	완료	AuthTokenStoragePolicy, AndroidKeyStoreAuthTokenCipher, legacy token migration
Keystore 실패 분류	완료	AuthTokenCipherFailure, AuthTokenReadResult, structured migration failure
Task surface domain 합성	완료	BuildTaskSurfaceListUseCase, BuildTaskSurfaceDateTodosUseCase, ObserveTaskSurfaceSummariesUseCase
TaskSurface 테스트 matrix 확대	완료	TaskSurfaceUseCasesTest의 filter/sort/date/override 조합
Todo sync 필드 정책 명확화	완료	TodoSyncFieldPolicy, NetworkTodoMutationPayload, TRD field policy
Todo due time sync	완료	dueTimeMinutes push/pull/fallback
TodoRepositoryImpl collaborator 주입성	완료	TodoRepositoryCollaboratorModule, injectable collaborators
Workspace refresh single-flight	완료	RefreshWorkspaceUseCase in-flight guard
Workspace refresh cached policy	1차 완료	WorkspaceRefreshPolicy, forceRefresh, allowCachedResult
Friends stale offline UX	1차 완료	unavailable block, stale snapshot banner, retry
Assignment feed freshness 영속화	완료	DataStore feed refresh time + in-memory tracker + row freshness
Assignment freshness sign-out reset	완료	AssignmentFeedFreshnessTracker.clear(), clearAssignmentFeedRefreshTimes()
Domain error taxonomy	기반 완료	AppError, AppErrorException, AppErrorMapping
Reminder failure normalize	완료	ReminderRepositoryImpl.mapFailureToAppError()
Reminder 반복 정책 공용화	완료	ReminderRecurrenceCalculator를 Reminder/Todo reminder 양쪽에서 사용
Schema bump 문서화	완료	v11->v12 no-op migration 주석
Person visibility read-only 흐름	1차 완료	PersonVisibilityRepository, ObservedTodo, Friends inline 친구 할일, Calendar observed section
Room v13 visibility schema	완료	visibility_grants, observed_todos, observed_sync_state, migration/schema test
PR rework observability harness	완료	branch metrics 문서, PR body reconciliation, review thread/follow-up commit 검사
Auth cleanup 정책	완료	deleteUserScopedLocalData(), deleteByOwner(), sign-out tests

13. 남은 개선 후보

우선순위	개선 후보	현재 관찰	제안
P1	Category sync 정합성	Todo sync가 categoryId를 왕복하지만 Category table은 로컬 전용이고 FK가 있다	Category server contract, remote/local id 분리, 또는 unknown category fallback 정책 확정
P1	PersonVisibility sign-out cleanup	auth session이 없으면 화면 Flow는 비지만 visibility_grants, observed_todos, observed_sync_state row는 sign-out cleanup 대상에 없다	AuthRepositoryImpl.deleteUserScopedLocalData()에 person visibility DAO cleanup을 포함할지 정책/테스트 추가
P1	Todo reminder sync 결정	reminder trigger/repeat/lead는 여전히 로컬 전용이다	여러 기기 reminder 일관성이 필요하면 API/TRD/worker 정책을 확장
P2	AppError adoption 확대	taxonomy와 일부 repository 적용은 했지만 전체 UI 소비는 아직 균일하지 않다	주요 repository failure를 mapFailureToAppError()로 normalize하고 feature error UI 정책 통일
P2	Workspace refresh 영속 정책	5분 cached-result policy는 process memory 기준이다	DataStore/DB 기반 last refresh, WorkManager retry/backoff, network constraint 도입 검토
P2	Assignment freshness key lifecycle	feed refresh time이 DataStore key로 누적될 수 있다	오래된 friend/feed key prune 또는 TTL 정책 추가
P2	Keystore recovery UX/관측성	structured failure는 있지만 사용자 안내와 metric은 제한적이다	save/decrypt/migration failure에 대한 안전한 로그/metric과 recovery UX 추가
P2	Friends persistent offline cache 여부	stale UX는 in-memory snapshot만 유지한다	완전 오프라인 friends UX가 필요하면 Room cache와 stale timestamp 설계
P3	Todo sync observability	sync 실패는 FAILED/halt reason 중심이고 운영 metric은 제한적이다	mutation result, cursor lag, auth halt, retry count 관측성 추가
P3	DataStore assignment key 문서화	storage key escaping 규칙이 구현 내부에 있다	feed key format과 호환성 정책을 짧은 문서로 남김

14. 권장 로드맵

1. Category sync 정합성을 먼저 확정한다. 현재 가장 큰 데이터 무결성 리스크는 Todo가 remote categoryId를 받는데 Category 자체가 로컬 전용이라는 점이다.
2. PersonVisibility local cleanup 정책을 확정한다. observed todo는 개인정보 성격이 강하므로 sign-out, auth-required clear, revoke purge의 책임 경계를 테스트로 고정한다.
3. Reminder sync 범위를 제품적으로 결정한다. reminder가 기기별 로컬 기능이면 현재 정책을 UI/ 문서에 명시하고, 계정 기능이면 API를 확장한다.
4. AppError adoption을 화면까지 마무리한다. 인증 필요, 네트워크 실패, validation, local data missing이 화면별로 같은 재시도/문구 정책을 갖게 한다.
5. Workspace refresh를 운영 정책으로 확장한다. process memory skip 위에 persistent freshness, retry/backoff, WorkManager constraint를 얹는다.
6. Assignment freshness key lifecycle을 정한다. sign out 정리는 이미 있으므로 장기 계정 사용 중 key prune이 필요하지만 판단하면 된다.
7. Keystore failure 관측성을 추가한다. migration 실패율, save failure, decrypt failure를 안전한 방식으로 측정하고 recovery UX를 마련한다.

15. 주요 코드 위치

영역	파일
모듈 선언	settings.gradle.kts

영역	파일
domain repository 계약	core/domain/src/main/java/com/neo/yourtodo/core/domain/repository/*
task surface domain	core/domain/src/main/java/com/neo/yourtodo/core/domain/usecase/TaskSurface*, BuildTaskSurface*, ObserveTaskSurfaceSummariesUseCase
app error taxonomy	core/domain/src/main/java/com/neo/yourtodo/core/domain/error/*
reminder recurrence	core/domain/src/main/java/com/neo/yourtodo/core/domain/reminder/ReminderRecurrenceCalculator.kt
workspace refresh	core/domain/src/main/java/com/neo/yourtodo/core/domain/usecase/RefreshWorkspaceUseCase.kt
person visibility domain	core/domain/src/main/java/com/neo/yourtodo/core/domain/repository/PersonVisibilityRepository.kt, ObserveObservedTodosUseCase.kt, ObserveVisibilityGrantsUseCase.kt, RefreshPersonVisibilityUseCase.kt
Room database	core/database/src/main/java/com/neo/yourtodo/core/database/AppDatabase.kt
Room migrations	core/database/src/main/java/com/neo/yourtodo/core/database/AppDatabaseMigrations.kt
DAO	core/database/src/main/java/com/neo/yourtodo/core/database/dao/*
Entity	core/database/src/main/java/com/neo/yourtodo/core/database/entity/*
DataStore source	core/datastore/src/main/java/com/neo/yourtodo/core/datastore/source/*
token storage policy	core/datastore/src/main/java/com/neo/yourtodo/core/datastore/source/AuthTokenStoragePolicy.kt, AndroidKeyStoreAuthTokenCipher.kt, AuthTokenCipher.kt
Network module	core/network/src/main/java/com/neo/yourtodo/core/network/di/NetworkModule.kt
Person visibility network	core/network/src/main/java/com/neo/yourtodo/core/network/personvisibility/*
Todo repository facade	core/data/src/main/java/com/neo/yourtodo/core/data/repository/ToDoRepositoryImpl.kt
Todo repository collaborator DI	core/data/src/main/java/com/neo/yourtodo/core/data/di/ToDoRepositoryCollaboratorModule.kt
Todo repository internals	core/data/src/main/java/com/neo/yourtodo/core/data/repository/todo/*
Assignment repository/freshness	core/data/src/main/java/com/neo/yourtodo/core/data/repository/AssignmentRepositoryImpl.kt, AssignmentFeedFreshnessTracker.kt
Person visibility repository	core/data/src/main/java/com/neo/yourtodo/core/data/repository/PersonVisibilityRepositoryImpl.kt
App sync integration	app/src/main/java/com/neo/yourtodo/app/AppSyncViewModel.kt
Todo UI data consumption	feature/todo/impl/src/main/java/com/neo/yourtodo/feature/todo/impl/ui/ToDoListViewModel.kt, ToDoListUiMapper.kt
Calendar UI data consumption	feature/calendar/impl/src/main/java/com/neo/yourtodo/feature/calendar/impl/ui/CalendarViewModel.kt, CalendarMonthUiMapper.kt, CalendarToDoUiMapper.kt
Friends UI data consumption	feature/friends/impl/src/main/java/com/neo/yourtodo/feature/friends/impl/ui/FriendsViewModel.kt, FriendsUiState.kt, FriendsCommonComponents.kt
Push integration	app/src/main/java/com/neo/yourtodo/app/push/*
Todo reminder WorkManager	app/src/main/java/com/neo/yourtodo/app/todo/reminder/*

영역	파일
Calendar widget	feature/calendar/widget/src/main/java/com/neo/yourtodo/feature/calendar/widget/*
Sync field TRD	docs/TRD- Todo-Sync-MVP.md
Person visibility PRD/TRD	docs/PRD-Person-Todo-Visibility-MVP.md, docs/TRD-Person-Todo-Visibility-MVP.md
Token policy 문서	docs/auth-token-storage-policy.md
Friends TRD	docs/TRD-Friends-MVP.md
Profile/sign-out TRD	docs/TRD-Profile-Menu-MVP.md

16. 결론

최신 main의 개선 버전은 이전 리포트에서 지적한 많은 항목을 실제 코드로 해결했다. Todo repository는 DI 가능한 collaborator 구조가 되었고, Todo sync는 due time과 category id까지 확장됐다. Assignment freshness는 DataStore로 영속화됐고, Friends는 온라인 전용 정책 위에 stale snapshot UX를 갖췄다. Workspace refresh와 token storage도 운영 리스크를 더 잘 드러내는 구조가 됐다.

여기에 PersonVisibility가 추가되면서 앱은 “친구에게 받은 일”과 “친구가 보여주기로 한 일”을 data model부터 분리하게 됐다. AssignedTodo는 내 task surface에 들어올 수 있는 협업 항목이고, ObservedTodo는 읽기 전용 projection이다. 이 분리는 Friends/Calendar UX뿐 아니라 Room schema, Retrofit API, domain use case, repository contract에도 유지되어 제품 의미가 코드 경계로 고정된다.

이제 가장 중요한 다음 결정은 Category sync와 PersonVisibility local cleanup 정책이다. 특히 categoryId는 이미 Todo sync에 포함됐기 때문에 Category 자체의 sync/id 정책을 늦게 정할수록 데이터 정합성 비용이 커진다. Observed todo는 개인정보 성격이 강하므로 sign-out/auth-required/revoke 시 로컬 row를 언제 지울지 명확히 해야 한다. 그 다음으로는 Reminder sync 범위, AppError adoption, persistent workspace freshness, Keystore failure observability를 정리하면 현재 아키텍처는 더 단단한 운영형 구조로 갈 수 있다.